

# Multi-Query and Grouped-Query Attention

Sharing keys and values to shrink the KV cache · video 2:42

This lesson answers one question: how do modern models keep the KV cache small? Instead of one key and value head per query head (**multi-head**), query heads **share**: all of them (**multi-query**) or groups of them (**grouped-query**). Qwen2.5-0.5B's measured cache is 7× smaller than without sharing; a tiny model trained with shared keys and values loses almost nothing, while sharing them in GPT-2 after training breaks it.

## How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

## Concepts in this lesson

| CONCEPT                               | VIDEO SEGMENT | QUESTIONS | MASTERED                 |
|---------------------------------------|---------------|-----------|--------------------------|
| 1. Sharing keys and values            | ▶ 0:08-0:41   | 2         | <input type="checkbox"/> |
| 2. Qwen2.5, measured                  | ▶ 0:41-1:16   | 3         | <input type="checkbox"/> |
| 3. The cost: trained in, or bolted on | ▶ 1:16-2:12   | 3         | <input type="checkbox"/> |
| 4. Speed, and the code                | ▶ 2:12-2:33   | 2         | <input type="checkbox"/> |

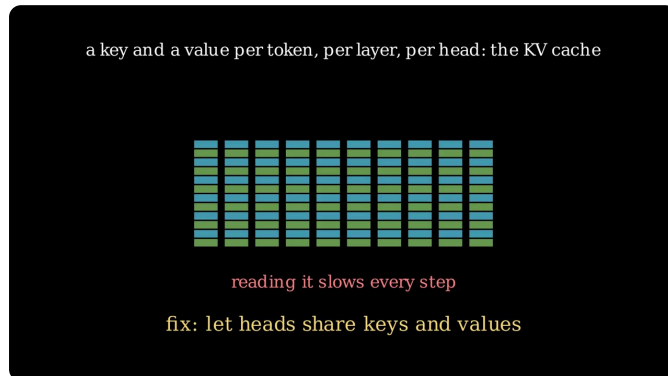
**Before you start:** LLMs in Practice episode 2 (KV cache) and Deep Dive episode 8 (one head, entry by entry). Code: `code/d09_gqa` (Qwen2.5-0.5B about 1 GB, GPT-2 about 500 MB, and three tiny models trained in a few minutes on a CPU).

## CONCEPT 1 OF 4

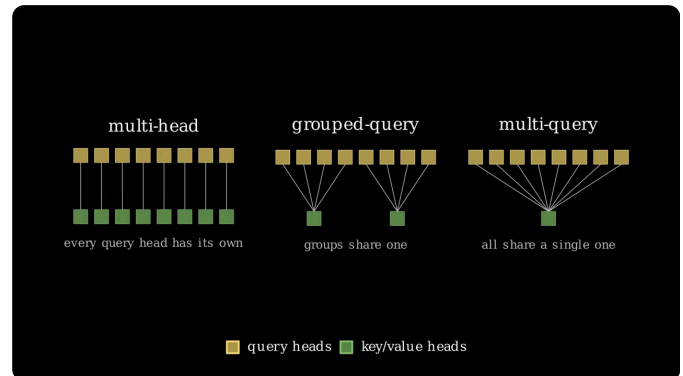
# Sharing keys and values

▶ Video 0:08–0:41

When a model writes, it keeps a key and a value for every token, in every layer and every head: the **KV cache**. On long texts it gets huge, and reading it slows every step. Most modern models let heads **share** their keys and values.



▶ 0:23 The KV cache: a key and a value per token, per layer, per head.



▶ 0:40 Multi-head: one K/V head per query head. Grouped-query: groups share one. Multi-query: all share one.

In classic **multi-head attention** (MHA), every query head has its own key and value head. In **multi-query attention** (MQA), all query heads share a single one. **Grouped-query attention** (GQA) sits in between: the query heads are split into groups, and each group shares one key/value head.

### KEY IDEA

The number of query heads stays the same; only the number of key/value heads, and so the cache, shrinks.

### Check 1: Sharing keys and values

Answer all 2 questions before looking at the answers. Pass mark: all correct.

#### 1.1 MULTIPLE CHOICE

A model has 32 query heads and 8 key/value heads. What is it?

- A. Multi-head attention
- B. Grouped-query attention, groups of 4
- C. Multi-query attention
- D. Grouped-query attention, groups of 8

#### 1.2 TRUE OR FALSE

“Multi-query attention uses a single query head.”

True ☐ False ☐

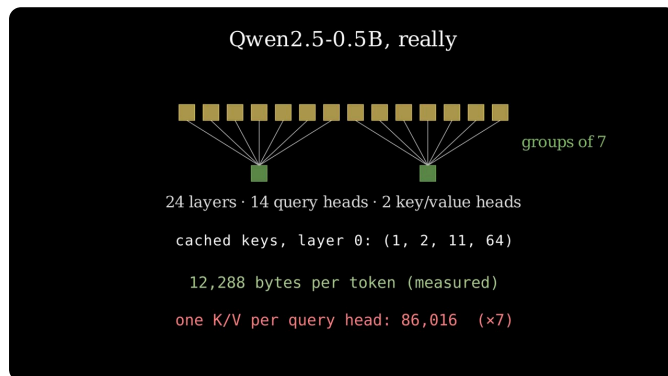
## Qwen2.5, measured

▶ Video 0:41-1:16

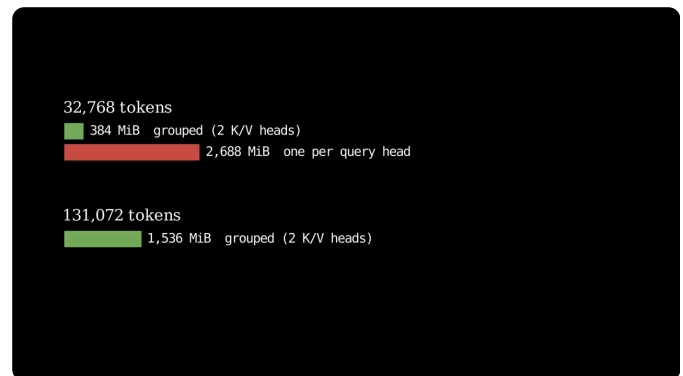
Qwen2.5-0.5B has 24 layers, **14 query heads** and only **2 key/value heads**: groups of 7. Its cached keys have shape (1, 2, tokens, 64): two heads, not fourteen. The cache per token is

$$2 \text{ (K and V)} \times 24 \text{ layers} \times 2 \text{ heads} \times 64 \text{ numbers} \times 2 \text{ bytes} = \mathbf{12,288 \text{ bytes}},$$

exactly what the code measures. With a key and value per query head it would be **86,016**, seven times more.



▶ 1:01 Qwen2.5-0.5B: 14 query heads share 2 key/value heads; 12,288 bytes of cache per token instead of 86,016.



▶ 1:14 At 32,768 tokens: 384 MiB instead of 2,688 MiB; at 131,072: 1,536 MiB instead of 10,752.

At 32,768 tokens that is **384 MiB** instead of 2,688 MiB; at 131,072 tokens, **1,536 MiB** instead of 10,752 MiB. Each query head still asks its own question; only the keys and values it looks up are shared: seven different questions asked of the same index.

### KEY IDEA

Cache per token = 2 × layers × key/value heads × head size × bytes per number; GQA cuts the key/value heads.

### Check 2: Qwen2.5, measured

Answer all 3 questions before looking at the answers. Pass mark: all correct.

#### 2.1 CALCULATION

How many bytes per token would Qwen2.5-0.5B's cache take in float32 (4 bytes per number)?

.....

#### 2.2 CALCULATION

A model has 32 layers, 8 key/value heads of 128 numbers, in bfloat16. How many bytes of cache per token? And with 32 key/value heads?

.....

#### 2.3 CALCULATION

About how many MiB does Qwen2.5-0.5B's cache take for a 1,000-token chat?

.....

## CONCEPT 3 OF 4

# The cost: trained in, or bolted on

▶ Video 1:16-2:12

The same tiny GPT (8 query heads, 4 layers) is trained three times for 2,000 steps, with 8, 2 or 1 key/value heads. Validation loss: **1.673**, **1.685**, **1.704**. The cache shrinks from 1,024 numbers per token to 256, then 128, and the model loses some parameters too (818,241 → 719,169 → 702,657). A loss of 0.03 for an 8× smaller cache.

But sharing has to be learned. Take GPT-2 and average its keys and values within each group, with no retraining: on 1,024 tokens of Shakespeare the loss jumps from **3.80** to **6.65** (4 K/V heads), 6.41 (2) and 6.52 (1). Each head was trained to expect its own keys. The grouped-query attention paper converts a trained model the same way, then fixes it with a short extra round of training (“uptraining”).

tiny GPT, 8 query heads, trained 2,000 steps

| K/V heads | val loss | cache / token |
|-----------|----------|---------------|
| 8         | 1.673    | 1,024         |
| 2         | 1.685    | 256           |
| 1         | 1.704    | 128           |

+0.03 loss for an 8× smaller cache

▶ 1:50 Trained from scratch: 8, 2 or 1 key/value heads give val loss 1.673, 1.685, 1.704, with an 8× smaller cache.

GPT-2: average keys and values per group, no retraining

|                        |      |
|------------------------|------|
| original, 12 K/V heads | 3.80 |
| 4 K/V heads            | 6.65 |
| 2 K/V heads            | 6.41 |
| 1 K/V head             | 6.52 |

loss on 1,024 tokens of Shakespeare  
each head expected its own keys  
fix: convert, then train a little more

▶ 2:10 GPT-2 with keys and values averaged per group, no retraining: loss jumps from 3.80 to 6.4–6.7.

### KEY IDEA

Built in from the start, sharing costs little; bolted onto a trained model, it needs extra training.

### Check 3: The cost: trained in, or bolted on

Answer all 3 questions before looking at the answers. Pass mark: all correct.

#### 3.1 SHORT ANSWER

Why does averaging GPT-2's keys within a group hurt so much, when training with shared keys barely hurts?

#### 3.2 CALCULATION

By what factor is the tiny model's cache smaller with 1 key/value head than with 8?

#### 3.3 TRUE OR FALSE

“In the episode's GPT-2 test, fewer key/value heads always gave a higher loss.”

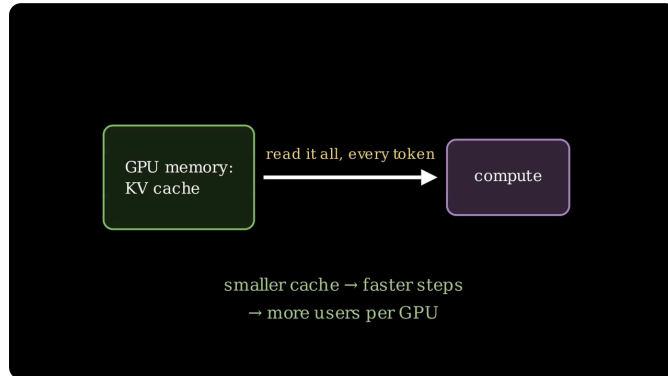
True ☐ False ☐

## CONCEPT 4 OF 4

# Speed, and the code

▶ Video 2:12-2:33

Generating a token means reading the whole cache from memory, so a smaller cache means faster steps and room for more users (bigger batches) on the same GPU.



▶ 2:21 Every generated token reads the whole cache from memory: smaller cache, faster steps, more users per GPU.

```
1 self.q = nn.Linear(D, H * HD)           # 8 query heads
2 self.k = nn.Linear(D, kv_heads * HD)      # fewer key heads
3 self.v = nn.Linear(D, kv_heads * HD)      # fewer value heads
4
5 k = k.repeat_interleave(H // kv_heads, dim=1) # one copy per query in the group
6 v = v.repeat_interleave(H // kv_heads, dim=1)
7 att = F.scaled_dot_product_attention(q, k, v, is_causal=True)
```

▶ 2:32 Key and value projections output fewer heads; each is repeated to match its group of queries.

In code, the key and value projections output fewer heads (`kv_heads × head size`), and each is repeated with `repeat_interleave` to match its group of queries. The rest of attention is unchanged.

### KEY IDEA

GQA is a two-line change to attention, and one of the main reasons long contexts are affordable.

## Check 4: Speed, and the code

Answer all 2 questions before looking at the answers. Pass mark: all correct.

### 4.1 HANDS-ON CODE

**Try it yourself.** Train the tiny model in `gqa.py` with 4 key/value heads (change the loop to `for kv in (4,):`). How many numbers per token does its cache hold?

```
for kv in (4,):
    ...
```

### 4.2 MULTIPLE CHOICE

With 8 query heads and 2 key/value heads, what does `k.repeat_interleave(4, dim=1)` produce from key heads `[A, B]`?

- A. `[A, B, A, B, A, B, A, B]`
- B. `[A, A, A, A, B, B, B, B]`
- C. `[A, B]`
- D. `[A, A, B, B]`

## ANSWERS

# Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

## 1. Sharing keys and values · rewatch ▶ 0:08-0:41

**1.1 B.**  $32 / 8 = 4$  query heads share each key/value head.

**1.2 False.** It keeps all its query heads; they share a single key and value head.

## 2. Qwen2.5, measured · rewatch ▶ 0:41-1:16

**2.1 24,576.**  $2 \times 24 \times 2 \times 64 \times 4$ ; twice the bfloat16 figure.

**2.2 131,072 (128 KiB); 524,288 (512 KiB) with 32.**  $2 \times 32 \times 8 \times 128 \times 2 = 131,072$ ; four times the key/value heads, four times the cache.

**2.3 About 11.7 MiB.**  $12,288 \times 1,000 / 2^{20} \approx 11.7$ .

## 3. The cost: trained in, or bolted on · rewatch ▶ 1:16-2:12

**3.1 GPT-2's query heads were trained against their own keys; the averaged keys are something none of them learned to read. A model trained with shared keys learns queries that work with them.** The weights adapt to the architecture during training; afterwards, they cannot without more training.

**3.2 8.**  $1,024 / 128 = 8$  numbers per token.

**3.3 False.** 4 heads gave 6.65, 2 heads 6.41, 1 head 6.52: all far worse than 3.80, but not in order. Without retraining, the damage is not graded.

## 4. Speed, and the code · rewatch ▶ 2:12-2:33

**4.1 512 numbers per token ( $2 \times 4$  layers  $\times$  4 heads  $\times$  16); val loss 1.680, between the 8-head 1.673 and the 2-head 1.685 (checked by running it).** The cache is linear in the number of key/value heads.

**4.2 B.** repeat\_interleave repeats each head in place, so query heads 0–3 use A and 4–7 use B: consecutive groups.